

Chapter Two : Context free Grammar

**Revision on a context-free
grammar and Pushdown Automata.**

Context-Free Languages

- The collection of languages associated with context-free grammars are called the *context-free languages*.
- They include all the regular languages and many additional languages.
- *Pushdown automata*, a class of machines recognizing the context-free languages and shows additional insight into the power of context-free grammars.

Context-Free Grammars

- A grammar consists of a collection of *substitution rules*, also called *productions*.
- Each rule appears as a line in the grammar, comprising a symbol and a string separated by an arrow.
- The symbol is called a *variable* represented by capital letters.
- The string consists of variables and other symbols called terminals. The terminals are analogous to the input alphabet and often are represented by lowercase letters, numbers, or special symbols.
- One variable is designated as the *start variable*. It usually occurs on the left-hand side of the topmost rule.

Context-Free Grammars

- A context-free grammar, G_1 .

$$A \rightarrow 0A1$$

$$A \rightarrow B$$

$$B \rightarrow \#$$

- For example, grammar G_1 contains three rules.
- G_1 's variables are A and B , where A is the start variable.
- Its terminals are 0 , 1 , and $\#$.

Context-Free Grammars

- For example, grammar G_1 generates the string 000#111.
- The sequence of substitutions to obtain a string is called a *derivation*.
- A derivation of string 000#111 in grammar G_1 is

$A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000A111 \Rightarrow 000B111 \Rightarrow 000\#111$

Context-Free Grammars

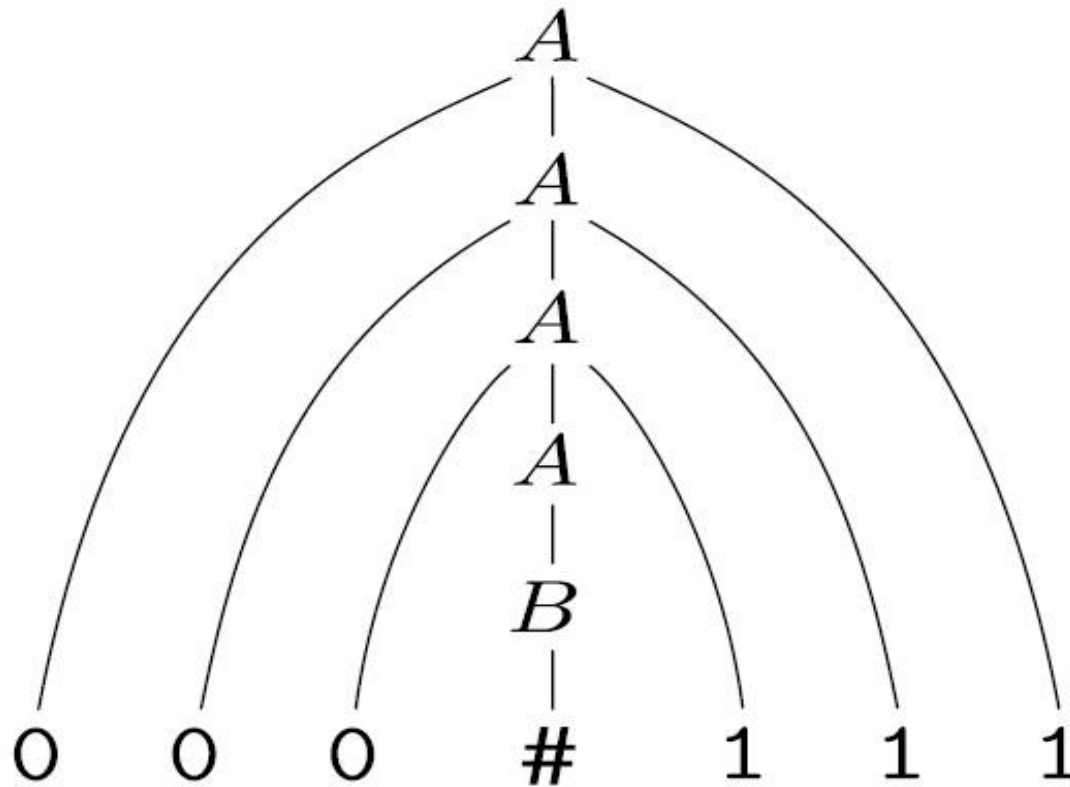


FIGURE 2.1

Parse tree for 000#111 in grammar G_1

Chapter Two : Context free Grammar

A context-free grammar

- Gives a precise syntactic specification of a programming language.
 - The design of the grammar is an **initial phase** of the design of a compiler.
 - A grammar can be directly converted into a parser by some tools.
-

Chomsky Normal Form

DEFINITION 2.8

A context-free grammar is in *Chomsky normal form* if every rule is of the form

$$A \rightarrow BC$$

$$A \rightarrow a$$

where a is any terminal and A , B , and C are any variables—except that B and C may not be the start variable. In addition, we permit the rule $S \rightarrow \varepsilon$, where S is the start variable.

- When working with context-free grammars, it is often convenient to have them in simplified form called Chomsky normal form.

Chomsky Normal Form

Theorem

- Any context-free language is generated by a context-free grammar in Chomsky normal form.

Proof Idea

- We can convert any grammar G into Chomsky normal form.
- First, we add a new start variable.
- Then, we eliminate all *ε -rules* of the form $A \rightarrow \varepsilon$.
- We also eliminate all *unit rules* of the form $A \rightarrow B$.
- In both cases we patch up the grammar to be sure that it still generates the same language.
- Finally, we convert the remaining rules into the proper form.

Chomsky Normal Form

Proof

- **First**, we add a new start variable S_0 and the rule $S_0 \rightarrow S$, where S was the original start variable.

Chomsky Normal Form

- **Second**, we take care of all ε -rules.
 - remove an ε -rule $A \rightarrow \varepsilon$, where A is not the start variable.
 - Then for each occurrence of an A on the right-hand side of a rule, add a new rule with that occurrence deleted.
 - In other words, if $R \rightarrow uAv$ is a rule in which u and v are strings of variables and terminals, we add rule $R \rightarrow uv$.
 - We do so for each *occurrence* of an A , so the rule $R \rightarrow uAvAw$ causes us to add $R \rightarrow uvAw$, $R \rightarrow uAvw$, and $R \rightarrow uvw$.
 - If we have the rule $R \rightarrow A$, we add $R \rightarrow \varepsilon$ unless we had previously removed the rule $R \rightarrow \varepsilon$.
 - Repeat these steps until we eliminate all ε -rules not involving the start variable.

Chomsky Normal Form

- **Third**, we handle all unit rules.
 - We remove a unit rule $A \rightarrow B$.
 - Then, whenever a rule $B \rightarrow u$ appears, we add the rule $A \rightarrow u$ unless this was a unit rule previously removed.
 - As before, u is a string of variables and terminals.
 - Repeat these steps until we eliminate all unit rules.
- **Finally**, we convert all remaining rules into the proper form.
 - We replace each rule $A \rightarrow u_1 u_2 \cdots u_k$, where $k \geq 3$ and each u_i is a variable or terminal symbol with the rules $A \rightarrow u_1 A_1$, $A_1 \rightarrow u_2 A_2$, $A_2 \rightarrow u_3 A_3$, $\dots$, and $A_{k-2} \rightarrow u_{k-1} u_k$. The A_i 's are new variables.
 - We replace any terminal u_i in the preceding rule(s) with the new variable U_i and add the rule $U_i \rightarrow u_i$.

CFG $\rightarrow$ Chomsky Normal Form

Example

- Let G_6 be the following CFG and convert it to Chomsky normal form by using the conversion procedure just given.
- The series of grammars presented illustrates the steps in the conversion.
- Rules shown in blue have just been added.
- Rules shown in red have just been removed.

CFG $\rightarrow$ Chomsky Normal Form

1. The original CFG G_6 is shown on the left. The result of applying the first step to make a new start variable appears on the right.

$$S \rightarrow ASA \mid aB$$

$$A \rightarrow B \mid S$$

$$B \rightarrow b \mid \varepsilon$$

$$S_0 \rightarrow S$$

$$S \rightarrow ASA \mid aB$$

$$A \rightarrow B \mid S$$

$$B \rightarrow b \mid \varepsilon$$

CFG $\rightarrow$ Chomsky Normal Form

2. Remove ϵ -rules $B \rightarrow \epsilon$, shown on the left, and $A \rightarrow \epsilon$, shown on the right.

$$S_0 \rightarrow S$$

$$S \rightarrow ASA \mid aB \mid \mathbf{a}$$

$$A \rightarrow B \mid S \mid \mathbf{\epsilon}$$

$$B \rightarrow b \mid \mathbf{\epsilon}$$

$$S_0 \rightarrow S$$

$$S \rightarrow ASA \mid aB \mid a \mid \mathbf{SA} \mid \mathbf{AS} \mid \mathbf{S}$$

$$A \rightarrow B \mid S \mid \mathbf{\epsilon}$$

$$B \rightarrow b$$

CFG $\rightarrow$ Chomsky Normal Form

3a. Remove unit rules $S \rightarrow S$, shown on the left, and $S_0 \rightarrow S$, shown on the right.

$$S_0 \rightarrow S$$

$$S \rightarrow ASA \mid aB \mid a \mid SA \mid AS$$

$$A \rightarrow B \mid S$$

$$B \rightarrow b$$

$$S_0 \rightarrow \mathbf{S} \mid ASA \mid \mathbf{a}B \mid \mathbf{a} \mid SA \mid AS$$

$$S \rightarrow ASA \mid aB \mid a \mid SA \mid AS$$

$$A \rightarrow B \mid S$$

$$B \rightarrow b$$

CFG $\rightarrow$ Chomsky Normal Form

3b. Remove unit rules $A \rightarrow B$ and $A \rightarrow S$

$S_0 \rightarrow ASA \mid aB \mid a \mid SA \mid AS$

$S \rightarrow ASA \mid aB \mid a \mid SA \mid AS$

$A \rightarrow \mathbf{B} \mid S \mid \mathbf{b}$

$B \rightarrow b$

$S_0 \rightarrow ASA \mid aB \mid a \mid SA \mid AS$

$S \rightarrow ASA \mid aB \mid a \mid SA \mid AS$

$A \rightarrow \mathbf{S} \mid b \mid ASA \mid \mathbf{aB} \mid \mathbf{a} \mid SA \mid AS$

$B \rightarrow b$

CFG $\rightarrow$ Chomsky Normal Form

4. Convert the remaining rules into the proper form by adding additional variables and rules. The final grammar in Chomsky normal form is equivalent to G_6

$$S_0 \rightarrow AA_1 \mid UB \mid a \mid SA \mid AS$$

$$S \rightarrow AA_1 \mid UB \mid a \mid SA \mid AS$$

$$A \rightarrow b \mid AA_1 \mid UB \mid a \mid SA \mid AS$$

$$A_1 \rightarrow SA$$

$$U \rightarrow a$$

$$B \rightarrow b$$

Chapter Two : Context free Grammar

- The use of CFGs has several advantages over BNF:
 - helps in identifying ambiguities.
 - a grammar gives a precise yet easy to understand syntactic specification of a programming language.
 - it is possible to have a tool which produces automatically a parser using the grammar.
 - a properly designed grammar helps in modifying the parser easily when the language changes.

Chapter Two : Context free Grammar

In a context-free grammar, we have:

- A finite set of terminals (in our case, this will be the set of tokens).
 - A finite set of non-terminals (syntactic-variables)
 - A finite set of productions rules in the following form :
 - $A \rightarrow \alpha$ where, A is a non-terminal and α is a string of terminals and non-terminals (including the empty string)
 - A start symbol (one of the non-terminal symbol).
-

Chapter Two : Context free Grammar

A context-free grammar is a 4-tuple (V, T, S, P) where

- (i) V is a finite set called the variables
- (ii) T is a finite set, disjoint from V , called the terminals
- (iii) P is a finite set of rules, with each rule being a variable and a string of variables and terminals, and
- (iv) $S \in V$ is the start variable.

If u, v and w are strings of variables and terminals, and $A \rightarrow w$ is a rule of the grammar, we say that uAv yields uwv , written $uAv \Rightarrow uwv$.

Chapter Two : Context free Grammar

Right-Linear Grammar

- In right-linear grammar, all productions have one of the two forms:

$$V \rightarrow T^* V \quad \text{or} \quad V \rightarrow T^*$$

- i.e., the left hand side should have a single variable and the right hand side consists of any number of terminals (members of T) optionally followed by a single variable.

Chapter Two : Context free Grammar

Left-Linear Grammar

- In left-linear grammar, all productions have one of the two forms:

$$V \rightarrow VT^* \quad \text{or} \quad V \rightarrow T^*$$

- i.e., the left hand side must consist of a single variable ,
and the right-hand side consists of an optional single
variable followed by one number of terminals.

Chapter Two : Context free Grammar

Example 1.

Construct right-and left-linear grammars for the language $L = \{a^n b^m : n \geq 2, m \geq 3\}$.

Right-Linear Grammar:

$$S \rightarrow aS$$

$$S \rightarrow aaA$$

$$A \rightarrow bA$$

$$A \rightarrow bbb$$

Left-Linear Grammar:

$$S \rightarrow Abbb$$

$$S \rightarrow Sb$$

$$A \rightarrow Aa$$

$$A \rightarrow aa$$

Chapter Two : Context free Grammar

Derivation Trees

A ‘derivation tree’ is an ordered tree which the nodes are labeled with the left sides of productions and in which the children of a node represent its corresponding right sides.

Chapter Two : Context free Grammar

Definition of a Derivation Trees

Let $G = (V, T, S, P)$ be a CFG. An ordered tree is a derivation tree for G iff it has the following properties:

- (i) The root of the derivation tree is S .
- (ii) Each and every leaf in the tree has a label from $T \cup \{\lambda\}$.
- (iii) Each and every interior vertex (a vertex which is not a leaf) has a label from V .
- (iv) If a vertex has label $A \in V$, and its children are labeled (from left to right) $a_1, a_2, \dots, a_n$, then P must contain a production of the form

$$A \rightarrow a_1, a_2, \dots, a_n$$

- (v) A leaf labeled λ has no siblings, that is, a vertex with a child labeled λ can have no other children.

Chapter Two : Context free Grammar

Right Most / Left Most / Mixed Derivation

aababbb

Consider the grammar G with production

$$\left\{ \begin{array}{l} 1. S \rightarrow aSS \\ 2. S \rightarrow b \end{array} \right\}$$

Chapter Two : Context free Grammar

Left Most Derivation

$$S \xRightarrow{1} aSS$$

$$\xRightarrow{1} aaSSS$$

$$\xRightarrow{2} aabSS$$

(Left Most Derivation)

$$\xRightarrow{1} aabaSSS$$

$$\xRightarrow{2} aababSS$$

$$\xRightarrow{2} aababbS$$

$$\xRightarrow{2} aababbb$$

Chapter Two : Context free Grammar

Right Most Derivation

$$\begin{array}{lcl} S & \xRightarrow{1} & aSS \\ & \xRightarrow{2} & aSb \\ & \xRightarrow{1} & aaSSb \\ & \xRightarrow{1} & aaSaSSb & \text{(Right Most Derivation)} \\ & \xRightarrow{2} & aaSaSbb \\ & \xRightarrow{2} & aaSabb \\ & \xRightarrow{2} & aababb \end{array}$$

Chapter Two : Context free Grammar

Mixed Derivation

$$S \xRightarrow{1} aSS$$

$$\xRightarrow{2} aSb$$

$$\xRightarrow{1} aaSSb$$

$$\xRightarrow{2} aabSb$$

$$\xRightarrow{1} aabaSSb$$

$$\xRightarrow{2} aabaSbb$$

$$\xRightarrow{2} aababbb$$

(Mixed Derivation)

Chapter Two : Context free Grammar

Example 2

A grammar G which is context-free has the

Productions $S \rightarrow aAB$ acbab

$$A \rightarrow Bba$$

$$B \rightarrow bB$$

$$B \rightarrow c.$$

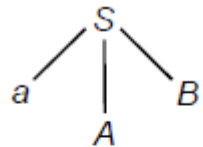
(The word $w = acbab$ is derived as follows)

$$S \Rightarrow aAB \rightarrow a(Bba)B \Rightarrow acbaB \Rightarrow acba(bB) \Rightarrow acbab.$$

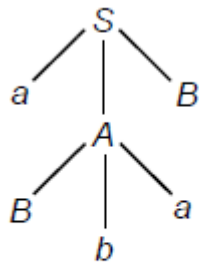
Obtain the derivation tree.

Chapter Two : Context free Grammar

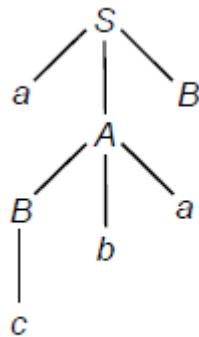
Parser Three



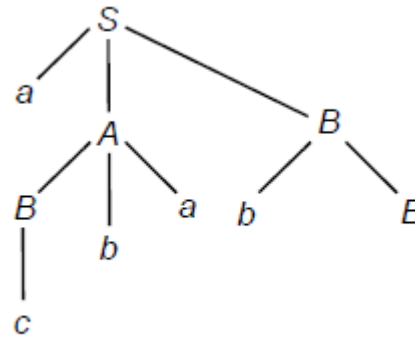
(a) $S \rightarrow aAB$



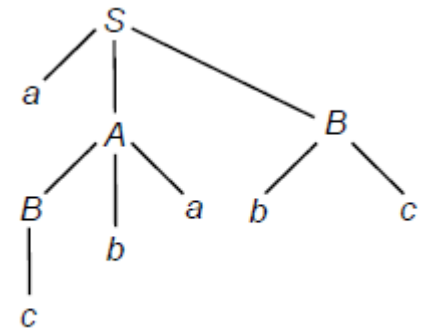
(b) $A \rightarrow Bba$



(c) $B \rightarrow c$



(d) $B \rightarrow bB$



(e) $B \rightarrow c$

Chapter Two : Context free Grammar

A CFG given by productions is :

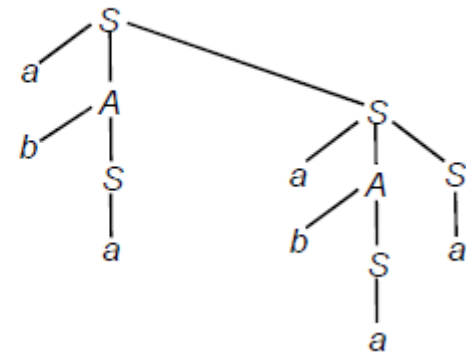
$$\begin{aligned} S &\rightarrow a, \\ S &\rightarrow aAS, \\ A &\rightarrow bS \end{aligned}$$

Obtain the derivation tree of the word $w = abaabaa$.

$w = abaabaa$ is derived from S as

$$\begin{aligned} S &\Rightarrow aAS \Rightarrow a(bS)S \Rightarrow abaS \Rightarrow aba(aAS) \\ &\Rightarrow abaa(bs)S \\ &\Rightarrow abaabaS \\ &\Rightarrow abaabaa \end{aligned}$$

The derivation tree is sketched below.



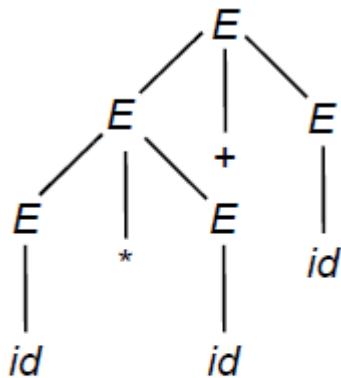
Chapter Two : Context free Grammar

Example 3 $id * id + id$ and $(id + id) * id$

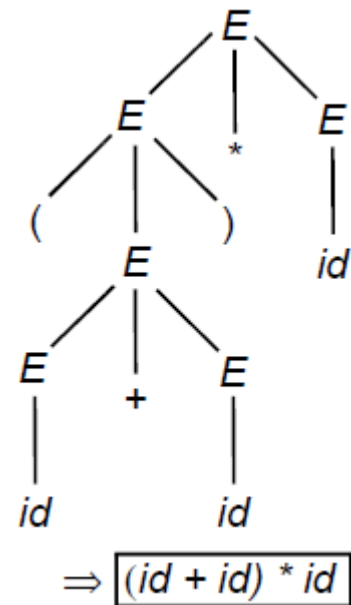
Given $G = (N, T, P, S)$ with $N = \{E\}$, $S = E$, $T = \{id, +, *, c\}$

P : 1. $E \rightarrow E + E$
and 2. $E \rightarrow E * E$
3. $E \rightarrow (E)$
4. $E \rightarrow id$

Obtain the derivation tree.



$\Rightarrow id * id + id$



$\Rightarrow (id + id) * id$

Chapter Two : Context free Grammar

Ambiguous Grammars / Ambiguous Languages

- Since derivation trees, leftmost derivations, and rightmost derivations are equivalent rotations, the following definitions are equivalent:

Definition: Let $G = (N, T, P, S)$ be a CFG.

A string $w \in L(G)$ is said to be “ambiguously derivable” if there are two or more different derivation trees for that string in G .

Definition: A CFG given by $G = (N, T, P, S)$ is said to be “ambiguous” if there exists at least one string in $L(G)$ which is ambiguously derivable. Otherwise it is unambiguous.

Chapter Two : Context free Grammar

Ambiguous Grammars / Ambiguous Languages

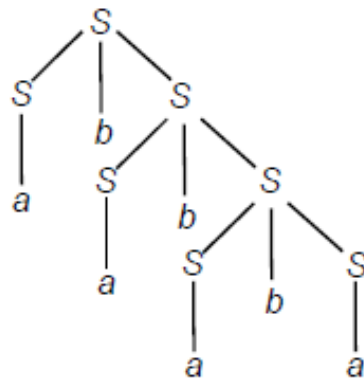
Show that the grammar $S \rightarrow S | S, S \rightarrow a$ is ambiguous.

In order to show that G is ambiguous, we need to find a $w \in L(G)$, which is ambiguous.

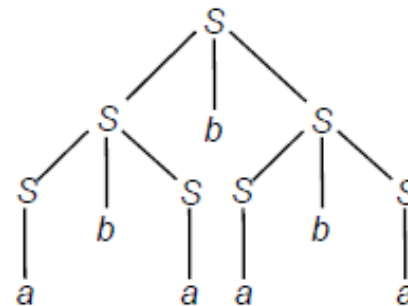
Assume

$w = abababa$.

The two derivation trees for $w = abababa$ is shown below in Fig. (a) and (b).



(a)



(b)

Therefore, the grammar G is ambiguous.